Capitolo 5

Livello Rete

5.1 Problemi Architetturali dello Stato Rete

Lo strato di rete si occupa di interconnettere tra loro i diversi collegamenti punto-punto, in modo da poter mettere in comunicazione tra loro anche dispositivi geograficamente lontani. Le informazioni che viaggiano nei pacchetti sono trasmesse dal mittente alla destinazione attraversando tutti i nodi della rete che si trovano sul tragitto congiungente questi due, I pacchetti sono interamente ricevuti da ogni nodo prima di essere ritrasmessi: (**store and forward**).

La progettazione del livello di rete deve tener presente i servizi da fornire al livello superiore (livello di trasporto) rendendoli trasparenti il più possibile e tra loro omogenei. Ci sono due scuole di pensiero per realizzare quanto appena detto: coloro che sostengono che la rete debba fornire un servizio orientato alla connessione e chi invece dice il contrario. Quelli che sostengono la prima tesi, fanno riferimento all' esperienza delle reti telefoniche e vorrebbero che la rete fornisse un servizio affidabile, in cui la QoS è un fattore fondamentale: un rete senza connessioni non potrebbe fornire un servizio di qualità (soprattutto per applicazioni Real Time). La seconda corrente di pensiero fa invece capo a coloro che hanno sviluppato Internet nel tempo, per i quali l' unica preoccupazione del livello di rete dovrebbe essere quella di accettare e inoltrare i pacchetti, ognuno completo di indirizzo destinazione e mittente, perché potrebbe prendere vie differenti durante il tragitto (a causa degli algoritmi di routing), mentre operazioni quali l' ordinamento e il controllo di flusso non dovrebbero appartenere a questo livello in quanto già implementate a livello superiore.

Le **reti non orientate** alla connessione sono anche dette reti a **Datagram**, perché l' unità di informazione è proprio il datagramma che si muove tra i nodi della rete: come detto poco innanzi, esso incorpora sempre gli indirizzi del destinatario e del mittente, ognuno di 32 o 128 bit a seconda che si usi IPv4 o IPv6. In queste reti, inoltre, poiché ogni datagramma è inoltrato in maniera indipendente dagli altri, i nodi che si trovano sul percorso attraversato (router) non mantengono alcuna informazione sullo stato delle connessioni, rendendo al quanto difficile la Qualità del Servizio e un eventuale controllo della congestione. Di contro, in caso di malfunzionamento di un router, non ci sarebbero grossi problemi sulla connessione in atto, se non la perdita dei datagrammi presenti nel router al momento del malfunzionamento.

5.2 Protocollo IPv4 e Indirizzamento

Per trasferire i pacchetti sulla rete c'era bisogno di un protocollo che gestisse questa operazione individuando univocamente mittente e destinazione, così ven- ne proposto **Internet Protocol**. IP è un protocollo connectionless e inaffidabile in quanto:

- tratta ogni pacchetto in maniera indipendente;
- non garantisce la consegna;
- non garantisce la correttezza dell' informazione trasportata;
- non richiede acknowledgment da alcun host, si tratti del destinatario o di nodi intermedi.

Ogni tipologia di rete ha una propria lunghezza massima di dati trasportabili da un frame, detta MTU (Maximum Transmission Unit), quindi può succedere che i dati provenienti dai livelli superiori debbano venire frammentati e trasmessi in pacchetti diversi.

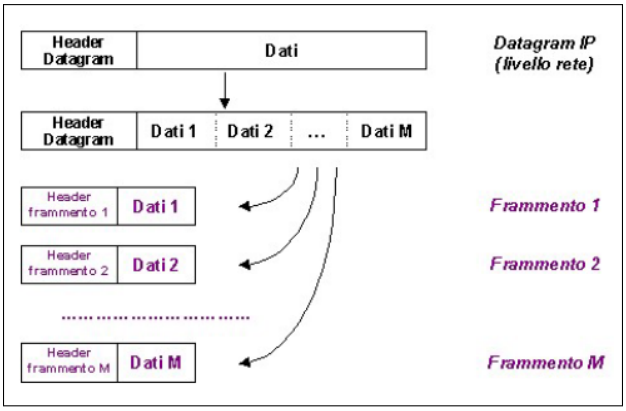

Figura 5.1: Frammentazione dei Pacchetti IPv4

Il fatto che IP sia un protocollo non orientato alla connessione e che un messaggio è suddiviso in un certo numero di pacchetti, implica che possa capitare che ciascuno di essi segua sulla rete un percorso diverso; in più, i pacchetti possono arrivare in un ordine diverso da quello in cui sono stati inviati, toccherà quindi a un protocollo di livello superiore rimetterli nell'ordine corretto.

5.2.1 Pacchetti IPv4

Il protocollo non solo definisce come i dati si spostano su Internet, ma anche come deve essere formata 'l'unità informativa' che effettivamente viene spostata su Internet: il pacchetto IP.

Un pacchetto IP è come un pacco una lettera con una busta che indica le informazioni sull'indirizzo, i dati contenuti al suo interno e attraverso quale altro protocollo di livello superiore essi devono essere interpretati. L'intestazione del pacchetto fornisce le informazioni necessarie per instradare il pacchetto verso la sua destinazione e in IPv4 può essere lunga fino a 24 bytes, con un minimo di 20 bytes: include, tra l'altro, l'indirizzo IP di origine, l'indirizzo IP di destinazione e le informazioni sulla dimensione dell'intero pacchetto.

L'altra parte fondamentale di un pacchetto IP è la componente dati, che può variare in termini di dimensioni. I dati all'interno di un pacchetto IP sono il contenuto che viene trasmesso. Il pacchetto IPv4 è allineato a 32 bit e la sua struttura, in particolar modo quella relativa al formato dell' header e delle sue componenti, può essere osservato in figura

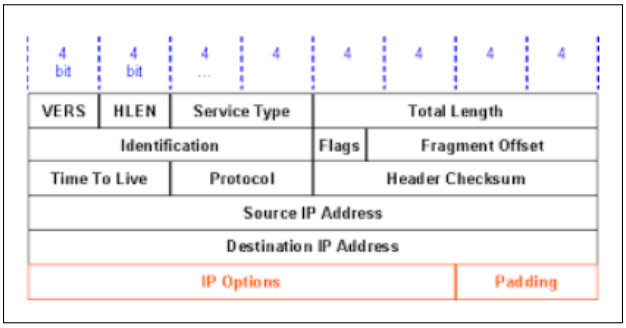

Figura 5.2: Intestazione IPv4

Vediamo ora i significati dei vari campi:

- **Version**: specifica la versione di IP utilizzata. La versione corrente del protocollo IP è di 4.
- **Length**: indica la lunghezza dell' intestazione del datagramma, misurata in parole di 32 bit.

- **Type of Service**: contiene cinque sottocampi che sono utilizzati per la QoS; essi specificano il tipo di precedenza, il ritardo, la velocità di trasmissione e l'affidabilità desiderata per quel pacchetto. Le impostazioni predefinite per questi cinque sottocampi sono la precedenza di routine, il ritardo normale, la normale velocità di trasmissione e la normale affidabilità.
- **Total Length**: specifica la lunghezza del datagramma comprensiva di intestazione e dati (misurata in byte).
- **Identification**: contiene un intero univoco che identifica il datagramma
- **Flags**: sono 3 bit utilizzati per il controllo del protocollo e della frammentazione dei datagrammi:
  - **Reserved**: sempre settato a 0.
  - **Don't Fragment**: di default settato a 0; se settato a 1 il pacchetto non viene frammentato, ma se sulla sua strada incontra un router/host che non prevede la non-frammentazione, allora viene scartato.
  - **More Fragment**: se settato a 0 indica che il pacchetto non è frammentato o è l'ultimo frammento del pacchetto originario; pertanto tutti gli altri suoi frammenti hanno il bit MF settato a 1.
- **Fragment Offset**: indica lo scostamento di questo frammento nel datagramma originale misurato in byte.
- **Time To Live**: è il tempo di vita (TTL) del pacchetto, in modo da evitare la sua persistenza indefinita sulla rete nel caso in cui non si riesca a recapitarlo al destinatario. Esso inizialmente misurava i "secondi di vita" del pacchetto, mentre ora rappresenta il numero di "salti" da nodo a nodo nella rete: ogni router che riceve il pacchetto prima di inoltrarlo ne decrementa il valore.
- **Protocol**: specifica il tipo di protocollo di livello superiore contenuto nel pacchetto; la lista dei codici di tali protocolli è mntenuta dalla IANA.
- **Header Checksum**: è un campo usato per il controllo degli errori dell'header.
- **Source/Destination IP Address**: indicano l'indirizzo IPv4 associato all'host del mittente/destinatario, formato da 32 bit suddiviso in 4 bytes.
- **Options**: informazioni facoltative e non molto usate, per usi più specifici del protocollo, come informazioni sui router o percorsi da seguire.

5.2.2 Indirizzamento IP

Siamo abituati a individuare un indirizzo IPv4 nella forma così come riportata di seguito:

208.67.220.220

Cioè composto da 4 numeri decimali separati da punti. Questa notazione testuale è pensata per renderne più semplice la lettura a noi umani. Internamente gli indirizzi IP vengono memorizzati in formato binario, cioè con una sequenza di 0 e 1.

La rappresentazione reale (come visto nell' intestazione del pacchetto IPv4 nel paragrafo precedente) viene fatta in binario ed è lunga 32 bit (divisi in 4 ottetti). Ad esempio l'indirizzo sopra corrisponde a:

11010000.01000011.11011100.11011100

Il numero totale di indirizzi IPv4 esistenti è 2^{32}, ossia 4294967296 (4 miliardi e 300 milioni circa), ma quelli realmente utilizzabili sono un po' di meno in quanto alcuni di essi sono riservati (ad esempio gli indirizzi 0.0.0.0, 127.0.0.1, 255.255.255.255, la classe 192.168.0.1/16).

Originariamente lo schema per individuare un indirizzo IPv4 era a classi (**classfull**); con questo schema l'indirizzo è ad autoidentificazione: per capire la classe a cui apparteneva bastava identificare i primi 4 bit dell'IP (i più significativi).

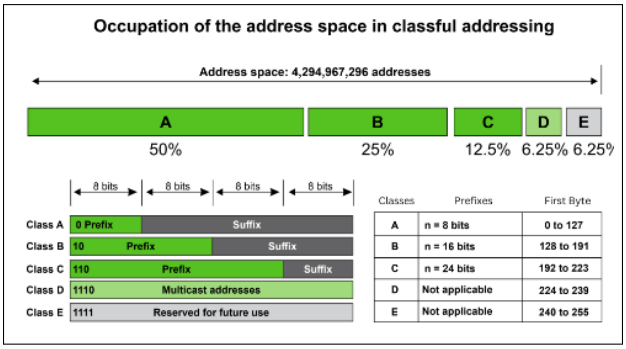

Figura 5.3: Indirizzamento Classfull

Quelle che si venivano a formare poi erano cinque classi nominate A,B,C,D,E e, in ciascuna classe, ogni indirizzo è stato diviso in due campi:

- l'indirizzo di rete (**netid**), che identifica la rete su cui si trova il computer e che costituisce numericamente una specie di prefisso;
- l'indirizzo del computer (**hostid**), che identifica il computer all'interno della rete e che costituisce numericamente una specie di suffisso.

L'indirizzamento a classi presenta diversi limiti dovuti soprattutto al numero di host gestibili dalle diverse classi. Se ad esempio si esauriscono gli indirizzi univoci resi disponibili da una classe occorre fare ricorso a un indirizzo di classe superiore, ma il cambiamento non è indolore.

Nella pratica, la crescita che negli anni ottanta ebbero le reti LAN ha portato all' esaurimento rapido degli indirizzi disponibili, quindi per risparmiare i prefissi di rete c'è stata necessità di escogitare altre tecniche come quella del mascheramento (**NAT**) per far si che IPv4 continuasse ad adempiere al suo ruolo.

Un' altra tecnica entrata in uso e che ha permesso di mantenere in auge IPv4 anche con l'alto numero di dispositivi connessi che abbiamo ora è quella soprannominata classless, conosciuta anche con l'acronimo CIDR: consiste nel mantenere l' IP suddiviso in due parti, ma questa volta il numero di bit riservati alla rete e di quelli dedicato all' host viene fatta in maniera dinamica. La modifica introdotta da CIDR consiste essenzialmente nell'utilizzare maschere di sottorete di lunghezza arbitraria, al contrario dell'indirizzamento con classi che ne prevedeva solo tre. Un esempio che tutti abbiamo visto almeno una volta è la classica rete privata spesso utilizzata in casa, che in notazione CIDR possiamo descrivere con: 192.168.0.0/24

Il numero 24 si riferisce al fatto che i primi 24 bit (quindi i primi tre ottetti) identificano la rete mentre la parte rimanente (8 bit) l'host. Ovviamente il numero non è necessariamente un multiplo di 8, rendendo a volte non immediato capire dove finisce la rete.

La possibilità di dedurre la rete da un indirizzo IP è cruciale per il funzionamento del sistema di indirizzamento IP: il prefisso viene infatti utilizzato dai router per capire verso quale direzione un pacchetto deve essere inoltrato per raggiungere la destinazione (attraverso la maschera di sottorete e la procedura di ANDing con l'indirizzo IP esso suppone il gateway, che di solito viene posto al primo o all'ultimo indirizzo del range). Un' altra cosa da tenere presente è la differenza tra IP pubblici e privati:

- gli **indirizzi IP pubblici** vengono utilizzati quando si interagisce con Internet;
- gli **indirizzi IP privati** operano sulla rete locale (LAN).

Su una tipica rete con configurazione semplice, il router utilizza un indirizzo IP pubblico per identificarla in maniera univoca sulla rete Internet. All'interno di questa rete può funzionare una varietà di dispositivi diversi. Il router (o altro device che ne fa le veci) assegna a ciascun dispositivo su questa rete un indirizzo IP privato (anch' esso univoco) in modo che i dati possano essere inviati al dispositivo che li sta effettivamente richiedendo.

Un indirizzo IP pubblico è l'indirizzo IP esterno (rivolto cioè verso la rete pubblica) assegnato al router dal provider di servizi Internet; essi possono essere di tipo statico o dinamico:

- un **indirizzo IP statico** è un tipo di indirizzo IP che non cambia. Quando un IP statico viene assegnato a un dispositivo, tale indirizzo rimane tale nel tempo e questo consente ad alcuni servizi online di funzionare in modo più uniforme e funzionale;
- un **indirizzo IP dinamico** può cambiare se necessario, come ad esempio quando un dispositivo si connette a una nuova rete. La maggior parte delle reti domestiche utilizza indirizzi IP dinamici perché sono più semplici e più economici da assegnare per i Provider di servizi Internet; essi, inoltre, possono risultare più protetti e offrire maggiore anonimato.

Ritornando alla divisione tra indirizzi pubblici e privati, si può dire che tutti gli indirizzi IPv4 sono pubblici se non rientrano nella categoria degli IP speciali (che abbia visto prima) o nel range degli IP privati. Nell' intervallo degli IP privati non importa se lo stesso indirizzo è assegnato anche in altre reti private, poiché essi devono essere univoci solo all'interno della stessa rete locale.

I range di indirizzi IP privati sono i seguenti:

- Intervallo IP privato di classe A: 10.0.0.0 – 10.255.255.255
- Intervallo IP privato di classe B: 172.16.0.0 – 172.31.255.255
- Intervallo IP privato di classe C: 192.168.0.0 – 192.168.255.255

5.3 Protocollo IPv6 e Indirizzamento

Il protocollo IPv6 è stato definito negli anni 90, a causa dell' esaurimento degli indirizzi IPv4, e la sua diffusione è iniziata a metà degli anni 2000; trattandosi comunque di un **protocollo molto diverso da IPv4**, e con retrocompatibilità limitata, **la sua crescita è stata molto lenta**. Tra i vari miglioramenti apportati esso doveva prevedere soprattutto un numero maggiore di indirizzi. Se IPv4 prevede indirizzi lunghi 32 bit, i**n IPv6 essi sono invece lunghi 128 bit, il quadruplo**, portando così ad avere uno spazio di indirizzamento estremamente vasto: si tratta infatti di 2^128 indirizzi. La maggiore lunghezza ha portato anche a utilizzare il sistema esadecimale per la rappresentazione testuale, componendo l' indirizzo con un totale di 32 caratteri raggruppati in 8 "parole" o gruppi da 4 caratteri ciascuno, separate dal simbolo due punti. Un indirizzo IPv6 è effettivamente lungo e difficile da ricordare, ma c'è la possibilità di comprimerlo togliendo le parti poco utili. Possiamo ad esempio rimuovere tutti gli zeri all'inizio di ciascun gruppo (**leading zeros**) e comprimere le parti composte solo da zeri (non più di una volta nello stesso indirizzo). Come conseguenza, lo stesso indirizzo IPv6 può essere scritto in più modi diversi ma equivalenti.

L' indirizzo ip seguente:

2001:0db8:85a3:**0000:0000**:8a2e:0370:7334

può essere scritto ad esempio

2001:0db8:85a3::8a2e:0370:7334

Il fatto che ci siano così tanti bit a zero nell'indirizzo non compresso non è un caso, in quanto in IPv6 l**a seconda parte di un indirizzo identifica infatti sempre l'host**, più precisamente chiamato ***interface ID*** perché lo stesso dispositivo può avere più interfacce di rete (es. WiFi ed Ethernet): esso è sempre lungo 64 bit e occupa perciò la seconda metà dell'indirizzo, dal bit 65 al bit 128 (una differenza rispetto a IPv4, dove la parte che separa rete e host è variabile); guardiamo il formato generico di tale indirizzo: Le tre parti principali della struttura dell' indirizzo le possiamo elencare come:

- I primi 3 bit rappresentano il range di indirizzi '**global unicast**' allocato da IANA (Internet Assigned Numbers Authority) in questo momento.
- Il **Subnet Prefix** (la "rete estesa"), che aggiunge al **Site Prefix** altri 16 bit fissi di sottorete (permettendo fino a 65 mila sottoreti per ogni rete, ossia per ogni organizzazione o "Site")
- L'**Interface ID** (l'Host), che occupa i rimanenti 64 bit, e che come abbiamo visto poc'anzi identifica la specifica macchina.

Dalla IANA è stato finora allocato 1/8 dello spazio di indirizzamento unicast IPv6, e precisamente quello che inizia per "001" (primo byte=001xxxxx, cioè gli indirizzi IPv6 = 2xxx::/3; quando finiranno si passerà a 3xxx::/3). In pratica questo significa che attualmente tutti gli indirizzi IPv6 pubblici iniziano con la cifra 2.

Esistono diversi tipi di indirizzi IPv6 consultabili a questo documento. Innanzitutto partiamo dal fatto che, come in IPv4, anche qui si usa la notazione CIDR, e una singola interfaccia viene indicata con il suffisso /128. Facciamo l' esempio con un paio di indirizzi 'particolari': quello non specificato e quello di loopback. Il primo, che in IPv4 corrisponde allo 0.0.0.0 e cioè qualsiasi indirizzo, viene scritto come:

::/128 (Unspecified)

Mentre il secondo, il nostro 'localhost', ossia il server presente sulla macchina sulla quale stiamo lavorando e che in IPv4 corrisponde al 127.0.0.1, viene indicato come:

::1/128 (Loopback)

Altri indirizzi speciali in IPv6 sono:

- gli **indirizzi riservati**, pari ad 1/256 dell'intero spazio di indirizzamento (hanno il primo byte = 0); sono usati per scopi di ricerca dell'IETF e per 'IPv4 address embedding'.
- gli **indirizzi locali** (i corrispondenti degli indirizzi privati in IPv4), che possono essere usati solo all'interno di ogni rete (o Site) e non vengono instradati verso l'esterno. Essi iniziano con i 9 bit: 1111 1110 1 (da FE8x::/9 a FEFx::/9) e sono anche detti 'unregistered' o 'nonroutable'. Questo sottogruppo è diviso a sua volta in due categorie:
  - i **Link-local Addresses**, che vengono sempre bloccati dai Router, e sono quindi locali solo a un segmento di rete (switched LAN) o ad una subnet. Vengono usati per la "automatic address configuration", usano le funzioni ND-Neighbor Discovery e per l'ARP. Hanno come decimo bit uno "0", per cui cominciano con FE8x, FE9x, FEAx e FEBx
  - i **Site-local Addresses**, che possono essere instradati dai Router di una organizzazione solo all'interno della rete privata (Site), quindi tra le sue subnet, ma non verso Internet; iniziano con FECx, FEDx, FEEx ed FEFx, avendo come decimo bit un "1"

In IPv6 l'idea è che ciascun dispositivo abbia assegnato almeno un indirizzo pubblico visibile su Internet. Per questo motivo un indirizzo di questo tipo è definito global unicast (GUA), indicazione che spesso si trova nella configurazione del proprio sistema operativo. Un esempio di indirizzo global è quello visto all' inizio.

Il fatto che ogni dispositivo abbia assegnato un indirizzo IPv6 pubblico è una differenza fondamentale rispetto a IPv4, dove, a causa della penuria di indirizzi, molti dispositivi sono costretti a usare indirizzi privati e ricorrere all'ausilio del NAT; come visto sopra, però, in IPv6 l'uso degli indirizzi privati risulta utile per svolgere servizi che devono rimanere interni, come può avvenire ad esempio nelle reti aziendali.

5.3.1 Prefix Delegation e Subnetting

Per gli ISP che supportano IPv6, il metodo più utilizzato per la configurazione delle CPE (Customer Premises Equipment) consiste nell'uso del protocollo DHCPv6 con Prefix Delegation: un router dell'operatore delega al router del cliente la possibilità di gestire un prefisso IPv6, di crearci delle sottoreti e di assegnare indirizzi pubblici a tutti i dispositivi in esse presenti. Secondo le raccomandazioni europee, nel nostro continente gli ISP dovrebbero delegare un prefisso statico /56 oppure /48 a ciascuna linea. Tali indirizzi prenderebbero la forma di

2001:db8:aaaa::/48

e

2001:db8:aaaa:1a00::/56

Abbiamo visto che l'interface ID di IPv6 occupa sempre la seconda metà dell'indirizzo, quindi il prefisso delegato può essere suddiviso in tante sottoreti (ad esempio /64). Prendendo in esame il caso di un prefisso /56 possiamo suddividere la rete locale in 256 sottoreti /64 ($2^{64-56} = 2^8$). Gli indirizzi assegnati ai dispositivi della rete vengono quindi scelti da una o più di queste sottoreti /64, permettendo di creare differenti sottoreti dedicate ad esempio a quella cablata, alla WiFi e alla rete ospite, permettendo scenari anche più complessi in ambienti aziendali; ognuna di esse (/64) permette di avere circa 18 miliardi di miliardi di indirizzi IPv6! Riferendoci ad esempio agli indirizzi sopra citati, le sottoreti /64 contenute nel prefisso sarebbero, in ordine:

2001:db8:aaaa:1a**00**::/64 2001:db8:aaaa:1a**01**::/64 . . . 2001:db8:aaaa:1a**fe**::/64 2001:db8:aaaa:1a**ff** ::/64

Tutti gli indirizzi IPv6 possono essere assegnati in maniera manuale o tramite protocollo DHCPv6; inoltre tutti i dispositivi che supportano IPv6 dispongono in automatico di un indirizzo link-local, attraverso il quale essi sono in grado di scoprire com'è fatta la rete in cui si trovano e di auto-configurarsi assegnandosi un indirizzo IPv6. Questo sistema si chiama **SLAAC** (Stateless Address Auto-Configuration) e non richiede di mantenere una lista di IP assegnati come farebbe un server DHCPv6. Gli indirizzi autoassegnati possono seguire due procedure:

- **assegnazione casuale**: come dice il nome stesso del processo l'indirizzo IPv6 viene generato in modo casuale;
- **sistema EUI-64**: l'interface ID viene generato a partire dall'indirizzo MAC della scheda di rete: questo è un indirizzo IPv6 facilmente riconoscibile perché contiene sempre al centro i caratteri ff:fe

Le versioni più recenti dei sistemi operativi implementano le **Privacy Extensions**, che consistono sostanzialmente nel cambiare periodicamente l'indirizzo auto-configurato per rendere più difficile il tracciamento e mantenere la privacy.

5.3.2 Altre Novità di IPv6

Nel rivedere il protocollo IP si è approfittato per fare anche ulteriori miglioramenti che non riguardano solo l'indirizzamento.

L'header dei pacchetti IPv6 è stato rivisto rimuovendo il campo checksum e prevedendo una lunghezza fissa per semplificare e velocizzare l'inoltro dei pacchetti sui router. L'header IPv6 è inoltre lungo il doppio rispetto a quello IPv4 (40 byte vs 20 byte) per via degli indirizzi più lunghi. L'header IPv6 è anche estensibile, cioè permette di definire funzionalità aggiuntive inserendo header 'a catena' come contenuto del pacchetto: essi però devono essere processati in modo sequenziale. Ogni Extensions Header è un intero multiplo di 8 ottetti per permettere l'allineamento naturale dei successivi. Un'altra novità è che IPv6 non include il supporto agli indirizzi di broadcast, ma prevede solo le tre macro-categorie di indirizzi: **unicast** (che abbiamo visto all' inizio), **anycast** e **multicast**; particolarmente fondamentali questi ultimi (ff00::/8).

Un indirizzo multicast viene infatti attribuito a un gruppo di interfacce di rete differenti (in linea di principio ognuna di esse appartenente ad altrettanti dispositivi distinti). I pacchetti inviati a un indirizzo multicast sono diretti a tutte le interfacce che fanno riferimento a questo indirizzo, ossia a quelle alle quali è stato attribuito.

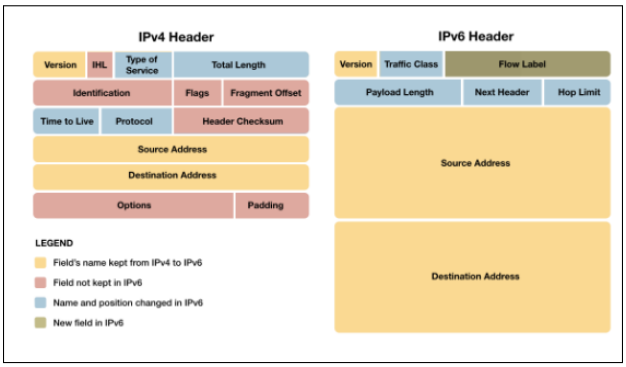

Figura 5.4: Confronto Header IPv4 e IPv6

Per quanto attiene agli indirizzi anycast, essi hanno le stesse caratteristiche di quelli unicast, ma vengono attribuiti a diverse interfacce di altrettanti nodi: lo scopo di un indirizzo anycast è quello di far sì che la rete consegni le informazioni al nodo del gruppo che risponde prima (ad esempio quello più vicino n base al protocollo di instradamento). Per la precisione, i pacchetti inviati a un indirizzo anycast dovrebbero raggiungere un'unica interfaccia di rete (la migliore in termini di costo).

Anche la sicurezza è stata curata in maniera migliore in IPv6 (ricordiamo infatti che in IPv4 essa è demandata a protocolli esterni che colmano il gap in questo campo): ci sono infatti due specifici extension header (abbiamo visto sopra di cosa si tratta) che forniscono servizi di questo tipo:

- **IP Authentication Header (AH)** ha il compito di fornire ai datagrammi IP integrità e autenticazione, ma non confidenzialità (questo per mantenere compatibilità con ambienti nei quali l'uso della crittografia è limitato). AH rende sicura la comunicazione tra due o più host, tra due o più gateway, tra host e gateway.

- **IP Encapsulation Security Payload (ESP)** a differenza del precedente questa modalità fornisce integrità, autenticazione e anche confidenzialità ai pacchetti. Come la precedente può essere usata per la comunicazione sicura tra due o più host, tra due o più gateway, tra host e gateway.

Sia AH che ESP possono essere utilizzati in due modalità: Transport Mode e Tunnel Mode; nel primo caso viene fornita protezione solo ai dati, mentre nel secondo caso viene fornita protezione all'intero datagramma IP.

Un concetto fondamentale sia per AH che per ESP è quello di Security Association (SA): una SA è una relazione, fra due entità in comunicazione, che identifica particolari condizioni di sicurezza.

5.4 Algoritmi di Routing

Nella sezione precedente abbiamo visto che il compito del router è quello di fornire due servizi principali:

1. Determinare il percorso ottimale

2. Trasportare le informazioni tra 2 reti diverse

Le reti sono spesso identificate con gli Autonomous System (AS), cioè in gruppi di network controllati e gestiti da fornitori di servizio Internet: i sistemi autonomi sono quindi di proprietà dei fornitori di servizi Internet (ISP). In altre parole, un AS è una rete individuale su Internet e ogni rete, o AS, ha un numero intero che la identifica (ASN), che viene assegnato dalla stessa autorità che rilascia gli indirizzi Internet. Tali numeri (ASN) sono quindi gli identificatori univoci rilasciati a un AS. L'identificatore consente a un AS di comunicare sia esternamente che internamente (in questo caso usa un ASN privato). I router che instradano i messaggi all'interno dello stesso AS sono spesso detti interior router e scambiano le informazioni tramite un protocollo, che è uguale su tutti i router, interno all' AS; mentre i router che instradano i messaggi tra AS diversi sono anche detti edge router o router di frontiera. In generale, ogni router mantiene in memoria una tabella di routing. Come minimo ogni entrata nella tabella deve contenere due elementi:

- Un **indirizzo di destinazione**: l' indirizzo IP del nodo di destinazione o della rete che lo ospita.

- Il **tipo d'indirizzo di destinazione**: può indicare che la destinazione è direttamente collegata al router oppure può indicare l'indirizzo di un altro router, anche questo collegato direttamente alla rete (next-hop router), ma che potrebbe portare a ulteriori reti.

Gli algoritmi di routing possono essere classificati in due grandi tipologie:

- **Statici**: in essi le tabelle di routing che vengono memorizzate sono predisposte da una persona (Network Administrator) e i valori di tali tabelle non cambiano per nessun motivo fino a quando il Network Administrator non li cambia.

- **Dinamici**: le tabelle vengono continuamente aggiornate e cambiate seguendo l' evoluzione della rete (caduta/inserimento di un nodo o di una network).

Per selezionare il percorso migliore, gli algoritmi scelgono secondo diversi criteri (**metriche**): per lunghezza del percorso, per banda passante, per affidabilità del link, per ritardo, per carico di rete. Tuttavia, due parametri universalmente accettati sono:

- **Hops**: numero di salti effettuati, cioè il numero di nodi attraversati lungo il cammino
- **Costo**: somma dei costi di tutte le linee attraversate (il costo di una linea è inversamente proporzionale alla sua velocità), e per costo si può intendere una qualsiasi delle metriche sopra elencate.

Proprio nella scelta del costo, ricade una delle caratteristiche tipiche degli algoritmi di routing, che si compongono di:

- **Ottimizzazione**: cioè l'abilità dell'algoritmo a scegliere la strada migliore (minimizzazione o massimizzazione della metrica).
- **Semplicità**: l'algoritmo deve essere funzionale ed efficiente con basso utilizzo delle risorse hardware.
- **Rapidità di convergenza**: quando ad esempio avviene un cambiamento nella rete, i routers distribuiscono, nel più breve tempo possibile, i messaggi di aggiornamento di tale evento affinché non si verifichino malfunzionamenti.
- **Scalabilità**: i routers devono adattarsi velocemente e accuratamente a una varietà di circostanze quali per esempio la caduta di una network: in questo caso i router devono scegliere il miglior percorso per tutti quei pacchetti che usavano la network caduta.
- **Robustezza**: a fronte di guasti hardware, uso intensivo delle risorse hardware e traffico elevato, l'algoritmo deve continuare a lavorare.

5.4.1 Distance Vector

Il primo algoritmo di routing dinamico che andremo a individuare e quello nominato Vettore delle Distanze (Distance Vector), basato sull' algoritmo di Belmann-Ford. L'idea base di questo algoritmo è che ogni nodo della rete mantenga al proprio interno una tabella che ne indica la distanza da ogni altro nodo della rete (il vettore delle distanze, per l'appunto). Supponiamo di inizializzare la rete alimentando tutti i nodi contemporaneamente (**cold start**) al tempo t_0 : in questo stato ogni nodo è caratterizzato da una conoscenza locale, ossia che esso sa solamente il proprio indirizzo e ignora totalmente la topologia della rete e la distanza dagli altri. La tabella di routing sarà quindi minima, caratterizzata da una singola voce: quella del nodo stesso. Prendiamo ad esempio una rete composta da 3 nodi e valutiamo cosa contiene il nodo A al 'cold start'.

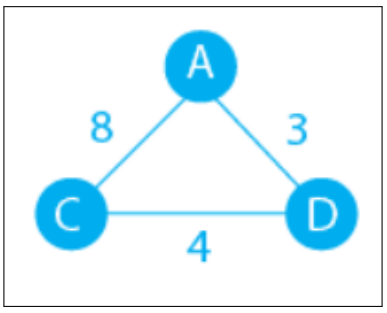

Figura 5.5: Rete Composta da Tre Nodi

Un vettore simile sarà presente sugli altri due nodi della rete, logicamente indicando rispettivamente C e D (al posto di A) in ognuno dei nodi. Successivamente, ognuno dei nodi invierà il proprio vettore delle distanze ai nodi a esso direttamente connessi; il vettore delle distanze presente nel nodo A sarà quindi il seguente

Ma cosa succede nel caso di un malfunzionamento su uno o più link ? Prati- camente viene fatto un merge dei vettori delle distanze tranne di quelli ricevuti dal link in errore, cosa può essere riassunta così:

- Il nodo scarta tutti i distance vector ricevuti da quel link
- Ricalcola la propria tabella mediante la fusione dei distance vector
- Distribuisce eventualmente il suo nuovo distance vector ai propri vicini

Il fenomeno della caduta di un link (o di un nodo) può innestare diversi problemi tipici di questo algoritmo come ad esempio:

- l' effetto rimbalzo (**Bouncing**) in cui, a causa di un possibile tempo di vulnerabilità tra l' indisponibilità del link e l' invio del distance vector del nodo che non sarà più disponibile, l'aggiornamento delle tabelle inizierà a far divergere l' algoritmo fermandosi solo al giungere del TTL dei pacchetti.
- La **divergenza**; si instaura un loop dovuto alla caduta di più link, ma, a differenza del precedente esempio, se i link caduti rendono effettivamente isolati i nodi non vi è alcuna possibilità per l'algoritmo di convergere ad uno stato stabile. Questo processo è anche conosciuto come conteggio all'infinito (**counting to infinity**) e può essere terminato solo attraverso una convenzione sulla rappresentazione di infinito con una distanza settata ad un valore maggiore del diametro della rete (ossia del percorso più lungo presente nella rete). Quando la distanza raggiunge questo valore, l' entry nella tabella di routing viene considerata infinitamente distante e, di conseguenza, irraggiungibile.

Da quanto detto, si capisce che l'effetto bouncing e il lungo tempo richiesto per il count to infinity sono effetti indesiderati a cui sono soggetti i protocolli di routing che fanno capo al distance vectors: vediamo alcune tecniche (tra le molte studiate) per minimizzare questi effetti.

Lo **Split Horizon** è la principale di questi e si basa su una precauzione molto semplice: se un nodo (A) sta instradando pacchetti per una destinazione (Z) attraverso un nodo intermedio (B), non è logico che quest'ultimo tenti di raggiungere (Z) attraverso A, per cui non c'è ragione per cui (A) debba annunciare a (B) che (Z) ha una distanza più breve da A. Nella pratica ciò si realizza facendo in modo che i nodi inviino differenti versioni del vettore (per tenere conto del fatto che alcune destinazioni sono instradate attraverso i suddetti links in uscita) invece di diffondere lo stesso distance vector su tutte le porte del router.

La versione di "**Split horizon con poison reverse**" e' più aggressiva: il vettore delle distanze dei nodi continua a includere tutte le destinazioni raggiungibili, ma essi imposteranno a infinito quelle distanze che sono instradate attraverso il link caduto; non si limitano cioè a escludere dal vettore le entries che sono per loro irraggiungibili, evitando di fatto che altri nodi possano erroneamente 'credere' di poter raggiungere quelle destinazioni per vie alternative.

5.4.2 Link State

La seconda tipologia di algoritmi di routing che vediamo è quella dello stato dei collegamenti (Link State). Questo tipo di algoritmi scambiano messaggi contenenti informazioni sullo stato dei collegamenti e consentono a ciascun router di apprendere l'intera topologia di rete: in tal modo ogni router riesce a calcolare la propria tabella di instradamento utilizzando l'algoritmo del percorso più breve.

Il principio di questa famiglia di algoritmi è semplice: non calcolano in maniera distribuita i percorsi migliori, bensì, ogni nodo mantiene una copia intera della mappa di rete, scambiando con ogni altro nodo della rete le informazioni sui suoi vicini, ed esegue un calcolo dei migliori percorsi per raggiungere gli altri. I concetti chiave per capire il Link State Routing sono tre:

- **Conoscenza dei nodi vicini**: al posto di inviare la propria tabella di routing, un router invia solo le informazioni sui suoi vicini. Ciascun router perciò invia a tutti i nodi della rete la sua identità e 'pesi' dei link che lo collegano direttamente ai suoi vicini.
- **Flooding**: ogni router invia le informazioni che detiene a tutti gli altri router della rete tranne a quelli a esso direttamente connessi; questo processo è conosciuto come flooding. Ogni router che riceve il pacchetto di informazione ne invia una copia ai propri vicini. A regime ogni router riceve una copia contenete le stesse informazioni.

- **Condivisione delle informazioni**: un router invia informazioni a ogni altro router solo quando vi è un cambiamento che modifica lo stato dei suoi link.

L' idea dei protocolli di routing 'link state' è di mantenere una copia sincronizzata della tabella sullo stato dei link in ogni nodi della rete. Abbiamo visto che per essere a regime è essenziale che tutte le copie presenti nei nodi siano identiche, altrimenti la coerenza degli instradamenti non potrebbe essere garantita. In realtà, durante alcune transizioni, potrebbero esserci dei nodi più aggiornati di altri; una situazione del genere potrebbe capitare, ad esempio, dopo un failure di un link.

L' algoritmo può essere suddiviso in quattro fasi:

1. **Inizializzazione**: ogni router della rete 'si informa' sui link che sono ad esso direttamente connessi. Le informazioni così apprese vengono inserite nella tabella del router

2. **Costruzione e distribuzione dei pacchetti**: Ogni router crea un pacchetto contenente la lista dei suoi vicini e i rispettivi costi di collegamento verso ciascuno di loro. Nell' intestazione del pacchetto, ogni router aggiunge la propria identità insieme a un numero di sequenza e un valore di durata (age): quest'ultimo viene utilizzato per garantire che i pacchetti non vaghino nella rete un periodo di tempo indefinito. Dopo il processo di costruzione, il pacchetto viene inviato nella rete tramite 'flooding', per raggiungere ogni nodo.

3. **Calcolo del percorso minimo**: ogni router usa le informazioni appena ricevute per creare un 'sink tree' e trovare il percorso minimo verso ogni nodo della rete; tipicamente viene applicato l' algoritmo di Dijkstra per trovare tali valori.

4. C**onsolidamento delle route**: in quest'ultima fase i percorsi calcolati vanno a formare la tabella che poi verrà salvata all' interno del dispositivo: essa è formata da tutti i percorsi ottimi verso ogni nodo della rete.

I vantaggi (1 - 2) e gli svantaggi (3 -4) di questa famiglia di algoritmi sono:

1. Hanno una convergenza abbastanza veloce in quanto ogni nodo ha bisogno di sapere solo lo stato dei link verso i nodi adiacenti, ed è molto importante nelle reti molto estese e con alto tasso di cambiamento di topologia.

2. Non soffre della problematica della divergenza, anche questo dovuto al fatto che le informazioni che si scambiano sono lo stato dei link circoscritti ai propri vicini.

3. É difficile da implementare correttamente in reti molto vaste in quanto non è scalabile come la famiglia del Distance Vector

4. Richiede un uso intensivo di risorse (memoria e tempo di CPU) per gestire le tabelle dell' intera rete e calcolare il percorso minimo a ogni cambiamento.

OSPF Uno dei più importanti algoritmi appartenente alla famiglia Link State è OSPF, acronimo che sta per Open Shortest Path First e ne esistono 3 versioni:

- OSPFv1: la prima versione, che ha gettato le basi per questo algoritmo molto usato, creato nel 1989, ma ormai quasi dl tutto sparito nella realtà.

- OSPFv2: in vigore dal 1998, usata nelle reti che hanno in uso IPv4.

- OSPFv3: l' ultima versione di OSPF, creata nel 2008, introduce l'uso di IPv6 pur rimanendo compatibile a IPv4.

I routers immagazzinano le informazione riguardanti la rete in pacchetti di tipo Link State Advertisements (LSAs) e poi le organizzano in una struttura chiamata Link State Database (LSDB). I routers 'inondano' la rete con LSAs finché tutti i nodi della regione OSPF hanno la stessa mappa di rete. Semplificando quindi il processo di come lavora OSPF in 3 operazioni principali, possiamo scrivere:

- Il primo passo è quello di collegarsi con gli altri routers connessi sullo stesso segmento.

- Quindi ogni router scambia un Link State Advertisements (LSAs) con i suoi vicini.

- Infine, ogni router, in maniera indipendente, calcola il percorso migliore (attraverso l' algoritmo di Dijkstra) verso ogni altra destinazione. Fatto ciò inserisce i percorsi migliori nella tabella di routing.

Una rete OSPF, soprattutto se molto estesa, può essere suddivisa in sottodomini chiamati aree. Un'area è una raccolta logica di reti OSPF, router e collegamenti che hanno la stessa identificazione di area. I router all'interno di un'area devono gestire i dati riguardanti la topologia della sola area a cui appartiene; questo fa sì che le dimensioni del DB siano ridotte rispetto alla conoscenza dell' intera rete.

Le aree limitano l'ambito di distribuzione delle informazioni di route. Il database dello stato del collegamento (LSDB, Link State Database) dei router della stessa area deve essere sincronizzato e deve essere esattamente lo stesso, tuttavia è possibile inviare un riepilogo dei percorsi tra aree diverse attraverso router prestabiliti. Ogni rete OSPF suddivisa in aree diverse deve utilizzare le regole seguenti:

- Deve esistere un'area backbone, che collega un insieme di aree indipendenti in un singolo dominio.
- Ogni area non backbone deve essere connessa direttamente all'area backbone.
- L'area della backbone non può essere frammentata in parti non collegate tra loro in condizioni di errore (disconnessione di un router o di un collegamento).

5.5 Congestione

La congestione della rete è il fenomeno che si verifica quando i nodi sorgenti immettono nella rete più pacchetti di quanti essa riesca a consegnare a destinazione; oltre che per quantità di pacchetti presenti sulla rete, essa può avere molteplici cause come:

- diversi flussi attraversano la stessa linea e i pacchetti che arrivano a un router non riescono a essere processati, si forma quindi una coda nel router
- l'esaurimento dei buffer nei router innesca un meccanismo che porta allo scarto dei pacchetti successivi
- la lentezza della CPU del router (nell'analizzare i pacchetti, la tabella di routing, o per attività di logging degli eventi) può portare a rallentamenti nella risposta ed esaurimento della memoria

L'insorgere di congestione nella rete, può essere riscontrato tramite due parametri che sono il ritardo e il throughput, e in mancanza di contromisure il suo effetto peggiora nel tempo.

Il ritardo in una rete è minimo e, in condizioni normali, è dovuto solo al ritardo di propagazione e al tempo di processamento nei router; man mano che il traffico aumenta, anche il ritardo cresce a causa delle ultime due cause sopra esposte, innescando per di più la rispedizione dei pacchetti scartati.

Quando la rete lavora in condizione di carico normale, il throughput cresce proporzionalmente al traffico; se il traffico supera una percentuale piuttosto alta della capacità del canale (intorno all' 85-90%), allora il throughput comincia a diminuire (le cause sono le stesse elencate sopra).

Esistono diverse tecniche sia per prevenire che per mitigare la congestione quando essa si verifica, e vengono applicate sia a livello rete che a livello trasporto (ad esempio nel protocollo TCP). A livello rete esse vengono suddivise in due macrocategorie:

- **Proattive** (o a ciclo aperto)
- **Reattive** (o a ciclo chiuso)

5.5.1 Controllo Proattivo

Queste tecniche cercano di prevenire la congestione. Il controllo viene fatto o dalla sorgente o dalla destinazione.

Tecniche di Ritrasmissione È la politica in cui viene posto l'accento sulla ritrasmissione dei pacchetti. Se il mittente ritiene che un pacchetto inviato sia andato perso o comunque sia arrivato danneggiato, il pacchetto deve essere ritrasmesso. Particolare attenzione però deve essere posta nella ritrasmissione in quanto può aumentare la congestione sulla rete. Per prevenirla, i timer di ritrasmissione devono essere progettati con molta cura, ottimizzando al massimo l'efficienza (tali tecniche vengono molto usate anche a livello trasporto, TCP).

Gestione della Finestra Scorrevole Il meccanismo di finestra scorrevole usato dal mittente può a sua volta avere più o meno influenza sulla congestione: la ripetizione selettiva sicuramente invia un numero minore di pacchetti sul canale rispetto al Go Back N.

Politiche di Eliminazione dei Pacchetti Le politiche di eliminazione dei pacchetti all' interno dei router è senz'alto una tecnica che aiuta sia a prevenire che a moderare la congestione. A volte i router sono costretti a eliminare preventivamente i pacchetti per evitare la congestione (ad esempio è possibile eliminare alcuni pacchetti di una trasmissione audio che poi verranno ricostruiti per interpolazione dal destinatario).

Politiche di Accesso Anche meccanismi studiati per la QoS possono aiutare a gestire la congestione: è il caso delle politiche di accesso per reti a circuito virtuale (orientate alla connessione). In queste reti i router coinvolti nella comunicazione devono indicare la disponibilità o meno delle risorse richieste dal circuito virtuale (il canale di comunicazione) che mittente e destinatario vogliono instaurare. In caso di risorse scarse (possibile congestione) meglio che i router neghino l'accesso alla rete.

5.5.2 Controllo Reattivo

Questa tecnica viene messa in atto quando la congestione si è verificata e bisogna attuare un qualche meccanismo che sia volto a eliminarla o, quantomeno, ad attenuarla.

Segnalazione al Mittente L'azione più semplice che può essere intrapresa da un router che sta riscontrando congestione è quella di inviare un 'segnale' al mittente per far sì che esso rallenti la sua trasmissione. Eventuali altri router intermedi coinvolti nella comunicazione non saranno informati. I choke packets sono un meccanismo in cui il router invia direttamente il pacchetto di segnalazione al mittente: quando si accorge della congestione, per diminuire il carico, il router rimanderà indietro solo i pacchetti choke. Questa tecnica, se usata in modo non attento, può contribuire a far aumentare la congestione. Nella suite TCP/IP questo può essere fatto usando il protocollo ICMP e il meccanismo Source Quench, che notifica al mittente che i buffers del router sono sovraccarichi. Quando il messaggio viene ricevuto dal mittente esso riduce la sua finestra di invio verso la destinazione 'in difficoltà', limitando il traffico in uscita. L' abuso di questo meccanismo potrebbe portare a un DoS, e infatti, nella moderna internet non viene quasi mai usato.

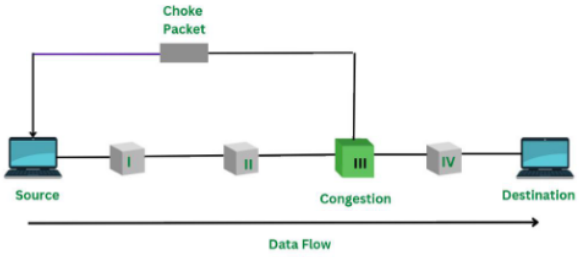

Figura 5.6: Segnalazione al mittente

Pressione all' Indietro Capita che la congestione non sia presente alla sorgente ma durante il tragitto. La pressione all'indietro è una tecnica in cui un nodo congestionato smette di ricevere pacchetti dal nodo che lo precede, ma ciò potrebbe spostare la congestione verso i nodi upstream (ossia nei nodi predecessori di quello che ha iniziato la backpressure). Il termine pressione all'indietro indica molto bene quello che succede, ossia la congestione si propaga, da nodo a nodo, nella direzione opposta al flusso di dati. Questa

tecnica può essere attuata solo in un circuito virtuale in cui ciascun nodo ha informazioni sul suo nodo che lo precede. Questa tecnica è conosciuta anche con il termine Choke packet hop-by-hop.

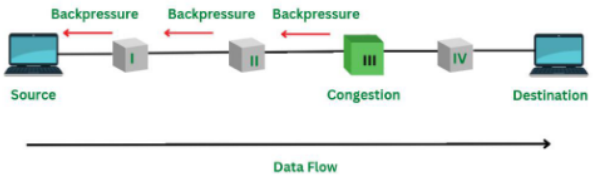

Figura 5.7: Pressione all'indietro

Segnalazione implicita Nella segnalazione implicita, non c'è comunicazione tra i nodi congestionati e la sorgente. La fonte suppone che ci sia una congestione in una rete in base ad alcuni eventi:

- quando il mittente invia diversi pacchetti e non viene ricevuta alcuna conferma prima della scadenza del timer;
- anche la ricezione di duplicati di conforme può essere indice di congestione nella rete.

Segnalazione esplicita Nella segnalazione esplicita, se un nodo riscontra una congestione invia esplicitamente un pacchetto alla sorgente o alla destinazione per informare della congestione. La differenza tra choke packet e segnalazione esplicita è che l'avviso di congestione fa parte dei pacchetti che trasportano i dati anziché crearne uno ad hoc come, per l'appunto, nel caso della tecnica choke packet. La segnalazione esplicita può avvenire sia in avanti che all'indietro:

- **Segnalazione diretta**: nella segnalazione diretta, l'avviso di congestione viene inviato nella direzione del flusso principale (quello in cui si riscontra il problema). La destinazione viene avvisata della congestione. Il destinatario in questo caso adotta politiche volte a prevenire ulteriori congestioni.
- **Segnalazione all'indietro**: nella segnalazione all'indietro, un avviso viene inviato nella direzione opposta al flusso dei dati. La fonte è avvisata della congestione e deve rallentare l'invio.

5.5.3 Leacky Bucket/Token Bucket

Due metodi per 'modellare' il traffico e ridurre la congestione sono l'algoritmo del **leaky bucket** e quello del **token bucket**. Il leaky bucket aiuta a controllare sia la quantità totale di traffico che la velocità con cui esso viene inviato alla rete, riuscendo anche a rilevare sia l'aumento graduale che i picchi repentini di errori (e quindi di pacchetti scartati) nel buffer. L'algoritmo funziona in maniera simile al modo in cui un secchio bucato trattiene l'acqua: l'acqua rappresenta i pacchetti (che possono arrivare fino a una capacità massima) che entra dall'alto nel secchio (input) e fuoriesce dal buco sul fondo (output). Se il flusso dati in entrata fosse troppo alto e i pacchetti riempissero il secchio, quelli in eccesso verrebbero considerati "non conformi" e quindi scartati. I pacchetti ritornano ad essere ammessi nel secchio non appena esso ritorna ad avere spazio svuotandosi.

L'algoritmo del leaky bucket è ideale per livellare il traffico di tipo burst ('sprazzi' di traffico al massimo della banda). Proprio come un buco sul fondo di un secchio che fa fuoriuscire l'acqua a una data velocità costante, così l'algoritmo del leaky bucket fa lo stesso con il traffico di rete, accumulando i pacchetti all'interno del secchio e lasciandolo fuoriuscire a velocità controllata attraverso il buco sul fondo.

Il buco può essere visto come settaggio di un particolare data rate della rete: il contenitore che perde modella il traffico in entrata per garantire che sia conforme, in uscita, al data rate settato. Si può implementare l'algoritmo Leaky bucket con un buffer gestito in modalità FIFO: la coda trattiene i pacchetti e non consente loro di passare se superano la larghezza di banda settata sulla rete in uscita, tuttavia, se il traffico continua ad arrivare e il buffer è pieno, l'algoritmo scarterà i pacchetti appena ricevuti; questo fenomeno, noto come "tail drop", avviene indipendentemente dall'importanza del pacchetto o dal flusso a cui esso appartiene.

L'algoritmo del leaky bucket modella il traffico a raffica in traffico a velocità fissa calcolando la media della velocità dati ma non tiene conto del tempo di inattività di un host inattivo. Al contrario, l'algoritmo del token bucket consente all' host, durante l'inattività, di accumulare credito per il futuro sotto forma di token, superando così il limite dell'algoritmo del leaky bucket. Nel token bucket si possono inviare pacchetti anche alla massima velocità se ci sono abbastanza token a disposizione (che vengono aggiunti a un rate costante, fino a un massimo previsto dal bucket, e vengono tolti ogni volta che si spedisce). Ci sono dei contro anche per questo secondo algoritmo: se, a esmepio, il bucket dei token è vuoto, i pacchetti dovranno attendere nuovi token, con conseguente aumento della latenza e potenziale perdita di pacchetti; inoltre è meno flessibile del leaky bucket quando si tratta di modellare il traffico di rete a causa della velocità fissa di generazione dei token che non può essere facilmente modificata per soddisfare i mutevoli requisiti della rete.

5.5.4 Load Shedding

Il **load shedding** o riduzione del carico è un' altra tecnica utilizzata per il controllo della congestione e interviene sui buffer del router. Sappiamo che i buffer sono utilizzati per conservare i pacchetti e poi instradarli verso la loro destinazione. La riduzione del carico è definita come un approccio che consiste nello scartare i pacchetti quando il buffer è pieno secondo una politica implementata a livello di collegamento. La scelta dei pacchetti da scartare è un compito importante e non può essere lasciato al caso, spesso ci viene in aiuto la conoscenza del traffico sulla rete:

- Priorità al più vecchio (**wine policy**): per il trasferimento di file, un vecchio pacchetto vale più di uno nuovo: questo perché eliminare un vecchio pacchetto potrebbe forzerà la ritrasmissione di altri pacchetti, inoltre il file risulterebbe inservibile in caso di 'buchi' nelle informazioni. Per questo tipo di applicazioni il pacchetto 'più vecchio è, meglio è'.
- Priorità al più fresco (**milk policy**): per il traffico multimediale, un nuovo pacchetto è più importante di uno vecchio (pensate alla comunicazione telefonica digitale, VoIP). In questo caso, quindi 'più è nuovo il pacchetto, più esso è importante'.

Come abbiamo appena visto la scelta dei pacchetti da scartare nei router può essere fatta **in base alle applicazioni** che girano sulla rete ma anche **in base alla priorità**. Il mittente può contrassegnare la priorità del pacchetto ad esempio usando il campo ToS (Type of Service) all' interno dell' header IP. A seconda della priorità settata, il pacchetto può essere selezionato o scartato. La priorità può essere assegnata in base al costo, all'algoritmo o ai servizi a cui appartiene.

Random Early Detection Un altro metodo molto usato (a livello rete) per decidere quale pacchetto scartare in caso di congestione è il **Random Early Detection** (RED); in realtà esso ha un comportamento proattivo in quanto cerca di prevenire la congestione. I punti essenziali dell'algoritmo sono i seguenti:

- per rilevare le condizioni di congestione si utilizza la dimensione dei buffer di uscita;
- per prevenire la congestione si marcano i pacchetti in uscita. Rispetto ad altri metodi utilizzano questi strumenti RED introduce alcune migliorie che ne aumentano le prestazioni:
- Non si considera la dimensione istantanea della coda, ma una forma di media pesata. Questo consente di ignorare picchi episodici, e trattare solo picchi di traffico persistenti.
- In base all' hardware e ai flussi di dati vengono scelte due soglie, una minima e una massima.
- Quando la lunghezza media della coda supera una certa dimensione contrassegnata dalla soglia minima, alcuni pacchetti vengono marcati: i pacchetti marcati vengono scelti a caso, e in un numero che cresce con la lunghezza media della coda.
- Oltre una certa soglia, tutti i pacchetti sono marcati.
- La marcatura del pacchetto è un'operazione in un certo senso simbolica: a seconda della tecnologia del router tale marcatura può consistere:
  - nella rimozione del pacchetto
  - nell'impostazione di un bit o campo nel pacchetto

In ogni caso si intende che la marcatura avrà un effetto di feedback sul mittente del pacchetto

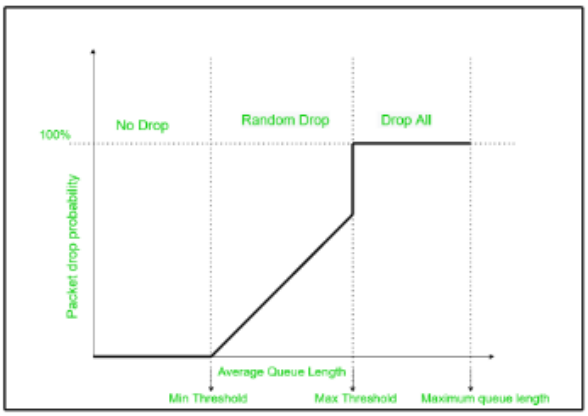

Figura 5.8: Random Early Detection